

Document Conversion Service

1. Introduction

This document serves as a sample text-only PDF for testing the Document Conversion and Splitting Service. It contains multiple paragraphs of text organized with headings and body content to validate the parsing and conversion pipeline.

The service is designed to accept PDF documents as input and convert them into either HTML or DOCX format. The conversion process is rule-based and deterministic, meaning text is carried across faithfully without any AI-based interpretation.

When the converted output exceeds a configurable size limit (default: 2 MB), the service automatically splits it into multiple ordered parts. Each part is clearly sequenced so the full set can be reassembled downstream.

2. Architecture Overview

The system follows Clean Architecture principles with four distinct layers: Domain, Application, Infrastructure, and API. The Domain layer contains entities, enums, exceptions, and interfaces with zero external dependencies.

The Application layer orchestrates the conversion pipeline through a `JobOrchestrationService` that drives the state machine end to end. It depends only on Domain interfaces, never on concrete infrastructure implementations.

The Infrastructure layer provides concrete implementations using PdfPig for PDF parsing, DocumentFormat.OpenXml for DOCX rendering, and Entity Framework Core with SQLite for persistence.

3. Conversion Pipeline

The conversion pipeline operates through several stages: document intake, parsing, format routing, rendering, splitting (if needed), and validation. Each stage transitions the job through well-defined status states.

The format router determines which output formats are valid based on document content. If the source PDF contains image XObjects, only HTML output is permitted. DOCX requests for such documents are rejected at intake with a fail-fast approach.

Splitting uses a sequential greedy accumulation algorithm. Elements are added to the current part in document order, and the actual rendered size is measured after each addition. Elements are never split or reordered.

Post-split validation ensures content integrity by comparing a canonical hash of the original document against the concatenated content of all parts. This approach avoids the pitfall of comparing raw byte sizes, which would be invalid due to per-part wrapper overhead differences.

4. Error Handling

The service handles several types of exceptions: `UnsupportedFormatException` for invalid format requests, `CorruptedFileException` for unreadable PDFs, `ScannedDocumentException` for image-only documents, and `EmptyDocumentException` for documents with no extractable content.

All exceptions extend a common `DocumentProcessingException` base class with a typed `ErrorCode` property, enabling the orchestrator to catch one type and route appropriately. Errors are persisted as part of the job history.